EconoCausal

Week 3 and Week 4 Action Plan, day by day

Dynamic Pricing via Double Machine Learning | Project ITS/DSML/1147 | Infotact Solutions

Plan window: Saturday 31 Oct to Monday 9 Nov 2026

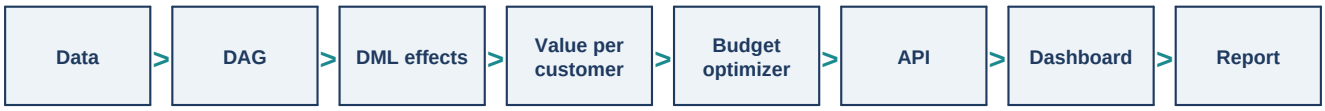
Date correction. Today is **Saturday 31 Oct**, not Wednesday 28 Oct as stated in the previous 13-day plan, so that plan's first three days have already passed. This plan restarts from today. The Final Review window opens on **5 Nov**, which gives you **5 build days** (31 Oct to 4 Nov) and then 5 days for documents and the review.

How the whole project fits together

Your README, project image and handbook describe the same project in three ways:

- **README (taken from the assigned brief):** a marketing director has a fixed discount budget and wants to spend it on customers whose behavior a discount will actually change. The brief's Week 3 is SciPy budget optimization and Week 4 is the API, drift checks and polish.
- **Project image:** a generic phase chart, including a Price / Sales / Revenue / Profit table.
- **Handbook:** how you are graded, which is mostly the Final Review rubric.

This plan follows the README, since it mirrors the brief. It also delivers the image's table as a **budget sweep table** (Budget / Customers / Spend / Incremental purchases / Incremental margin), so you cover both.



The one handoff file

Everything hangs on `data/processed/effects.csv` with columns `customer_id, tau_hat, tau_lo, tau_hi, p0_hat, e_hat, discount_amount, margin`. The optimizer reads it, the API serves it, and the dashboard can run on a mock copy of it. That is what lets people work in parallel.

The value formula (put it in your methodology doc)

- `p0 = m(X) - e(X) * tau(X)`: baseline purchase probability without an offer, taken from the DML nuisance models and clipped to [0, 1].
- Incremental margin of offering discount d: `tau * (margin - d) - p0 * d`. The second term is the cost of giving a discount to someone who would have bought anyway, which is the exact problem your README describes.
- Expected spend: `(p0 + tau) * d`. This is the spend you cap with the budget.

If your data has no margin, label the objective **purchase uplift**, not profit.

Roles

Split into three tracks: **A** (causal, optimizer, API), **B** (dashboard) and **C** (tests, evaluation, docs). If you are solo, do A, then B, then C each day, and use the cut list at the end.

Gate check: first 90 minutes today (everyone)

- `pip install econml` imports on Python 3.14. If not, use a 3.11 or 3.12 venv.
- `python -m pytest tests/ -v` is green.
- A working `dml_estimator.py` with 5-fold cross-fitting writes `effects.csv`. If Week 2 is incomplete, this is the minimum to build today.
- Ask your mentor whether the Mid Review was held, since its window closed on 27 Oct.

Build days (31 Oct to 4 Nov)

Day 1, Sat 31 Oct: Effects file and optimizer v1

Track	Tasks
A	Finish `dml_estimator.py` and export `effects.csv`. Write `budget_optimizer.py` v1 with the value and spend formulas and a 0/1 knapsack using `scipy.optimize.milp` (SciPy 1.9+). Add a greedy value-per-spend baseline to cross-check it.
B	Add the Vite proxy (`/api` to `localhost:8000`) and an `api.js` client. Build the Budget page controls and results table against a mock allocation JSON.
C	Write `src/evaluation/metrics.py`: nuisance MAE/RMSE, Qini/AUUC, uplift by decile, doubly robust policy value, budget utilization, cost per incremental conversion. Write `tests/test_optimizer.py`: budget never exceeded, zero budget gives an empty selection, negative-value customers are never picked, greedy and MILP agree closely.

Done when: `python -m src.optimization.budget_optimizer --budget 50000` prints a real selection summary and the tests pass.

Day 2, Sun 1 Nov: Constraints, comparison and Budget page

Track	Tasks
A	Add constraints: maximum customers, eligibility flag, segment caps, and a **conservative mode** that only targets customers whose lower confidence bound `tau_lo > 0`. Add the budget sweep (5-8 budgets). Add the policy comparison (no discount, random, blanket, causal) on a held-out fold using the doubly robust value, plus the true effect if your generator stored it.
B	Full Budget Allocation page: budget slider, constraint inputs, KPI cards (spend, utilization, customers, incremental margin), allocation table, sweep chart, policy bar chart, CSV export.
C	Test the constraints, and test that a bigger budget never gives a lower value. Write `reports/week3_report.md` and start `docs/methodology.md`.

Done when: the dashboard shows allocation, sweep and policy comparison from real numbers. Merge to main and tag `week3`.

Day 3, Mon 2 Nov: API and database

Track	Tasks
A	Build `src/api/main.py` with the six README section 15 endpoints. Add pydantic schemas, a file-size limit, clear error messages (missing columns, no overlap), CORS, and a SQLite `runs` table (run_id, time, config, summary, artifact path). Start it with `uvicorn src.api.main:app --reload`.
B	Wire the Data Quality page to `/data/validate`. Wire the Causal page to estimate and diagnostics (DAG, propensity overlap histogram, effect distribution, balance table). Build the Uplift page (Qini curve, decile uplift).
C	Write `tests/test_api.py` with `TestClient`: health, bad upload, estimate, optimize, unknown run returns 404. Write `tests/test_causal_pipeline.py`.

Done when: a click in the dashboard goes through the API, saves a run in SQLite, and the run can be fetched again.

Day 4, Tue 3 Nov: Drift, evaluation and reporting

Track	Tasks
A	Add `src/evaluation/drift.py` with PSI and KS checks per covariate, and expose it through the API. Save the DoWhy refutations (placebo treatment, random common cause) and a stability table across learners.
B	Build the Experiment Report page (method, assumptions, metrics, limitations, export). Add a plain-language summary under every chart. Add warning banners for poor overlap, drift and 'synthetic data'.
C	Fill the README section 13 results table with measured numbers only, stating the evaluation method. Finish `docs/methodology.md` and `docs/architecture.md`. Start the report skeleton and PPT outline.

Done when: every item in README section 13 appears somewhere in the app or the report.

Day 5, Wed 4 Nov: Freeze

- No new features. Bug fixes only.
- Someone who did not write the code does a clean clone and follows the README exactly.
- Add one script, `run_all.ps1`, that runs the pipeline, the tests, the API and the dashboard.
- Merge every member branch to main and tag `final-rc`. Docker is optional, so only add it if everything else is done.

Done when: the project runs on a clean clone with no help from the authors.

Review readiness (5 to 9 Nov)

Day	What to do
Thu 5 Nov	The Final Review window opens, so ask your mentor for a slot today and do not wait. Finish the final report (problem, data, DAG assumptions, method, results with intervals, policy comparison, limitations, fairness and privacy note), the PPT (following the six phases in your image), and the final README.
Fri 6 Nov	Record the demo: upload, quality, causal, uplift, allocate, export, report. Each member prepares answers for their own part.
Sat 7 Nov	Record and submit the social media video (5 marks). Do a timed dry run with a mock viva. Keep a screen recording as a fallback if the live demo fails.
Sun 8 Nov	Attend the review if it is scheduled, with the whole team present (5 marks).
Mon 9 Nov	Last safe day, since the window closes 10 Nov. Tag `final` and confirm submissions.

Viva questions to rehearse

- Why is correlation not causation?
- What does residualization do?
- Why do we use cross-fitting?
- What does the DAG assume, and what is overlap?
- Why does the value formula subtract p0 times d?
- What happens if there is hidden confounding?

Final Review rubric coverage

Handbook item	Where it is covered
Completion of requirements	Days 1-4
Frontend and backend integration	Day 3
Database implementation	SQLite run store, Day 3
API functionality	Day 3, with tests
Advanced features	Conservative targeting, budget sweep, drift checks, uncertainty intervals
Code quality and structure	Modules as in README section 11, tests, methodology doc
Individual contribution	Each member's own branch and track
Deployment and demo	`run_all.ps1`, clean-clone test, optional Docker
Final Viva	Days 6-8

If you fall behind, cut in this order

1. Docker or hosting.
2. Discount-level optimization (several discount amounts per customer).
3. Segment caps.
4. CausalForestDML, keeping LinearDML.
5. Extra dashboard polish.

Never cut: `effects.csv`, the budget optimizer, the policy comparison, the API (`/health`, `/causal/estimate`, `/optimize`, `/runs/{id}`), the Budget page, the report and the demo.

Every day

Each member makes at least one meaningful commit to their own branch, and the Team Lead merges to main. The Final Review needs a commit on every one of the 20 days before it. If there is already a gap in your calendar, tell your mentor now. Make no ROI claim your measured results do not support.

Tick-off tracker

Tick each line when its 'done when' condition is true, not when the work feels finished.

Day	Date	Done when	Done
Gate	Sat 31 Oct	econml imports, tests green, effects.csv exists, mentor asked about Mid Review	[ ]
1	Sat 31 Oct	Optimizer CLI prints a real selection summary; optimizer tests pass	[ ]
2	Sun 1 Nov	Dashboard shows allocation, sweep and policy comparison; week3 tagged	[ ]
3	Mon 2 Nov	UI to API to SQLite round trip works; API tests pass	[ ]
4	Tue 3 Nov	Every README section 13 item is in the app or report; results table filled	[ ]
5	Wed 4 Nov	Clean clone runs with no help; all branches merged; final-rc tagged	[ ]
6	Thu 5 Nov	Review slot requested; report, PPT and README final	[ ]
7	Fri 6 Nov	Demo recorded; each member has viva answers for own part	[ ]
8	Sat 7 Nov	Social media video submitted; timed dry run and mock viva done	[ ]
9	Sun 8 Nov	Review attended by the whole team (if scheduled)	[ ]
10	Mon 9 Nov	final tagged; submissions confirmed; review scheduled if not yet held	[ ]